GUÍA DE ESTUDIO · CAPACITACIÓN VELA LATAM

Desarrollando con Inteligencia

IA para Desarrolladores · Claude y Claude Code en todo el SDLC

Campo	Valor
Programa	«Desarrollando con Inteligencia — IA para Desarrolladores», Vela LATAM, Grupo 02, versión 4 (2026)
Instructor	Felipe Cuevas (Claude Certified Developer – Foundations)
Calendario	**Lunes 9 a viernes 13 de noviembre de 2026**, de 09:00 a 16:00 (hora de Chile), con almuerzo de 12:00 a 13:00
Sitio del curso	ffelipecuevasc.github.io/curso-ia-desarrolladores (contenido de cada unidad y guías de los talleres)
Formato	30 horas: 10 unidades de 3 horas (85 min de cátedra, 15 de receso y 80 de taller), dos por día, durante 5 días
Ecosistema	Anthropic: Claude y Claude Code, con licencias **Claude Team Standard**; JetBrains (Rider, WebStorm), .NET 10, xUnit, Docker, PostgreSQL 16, GitHub Actions, Python y Playwright
Caso integrador	**«Pedidos Legado → Pedidos Web»**: un sistema de pedidos en Delphi/DBISAM que, unidad a unidad, se convierte en un portal web en .NET y PostgreSQL
Marcas	★ imprescindible · ○ si hay tiempo

Antes de empezar.
1. **La fuente de verdad es el programa** (PDF de la ementa). Lo que esta guía agrega como «lectura propia» o «verificar en el curso» es interpretación nuestra.
2. **No llevar código ni datos de Rayen al entorno del curso.** Los laboratorios usan repositorios sintéticos y licencias del curso. El código fuente de Rayen es información **Secreta** (SGSI-PO-003) y los datos clínicos son **rojos**. Si queremos practicar con algo nuestro, primero hay que confirmarlo con el OSI.
3. La sección §13 resume el trabajo previo sobre Simplifique (la plataforma de Contmatic) y la Plataforma IA de Rayen. Sirve como contexto para las unidades 8 a 10, pero **no es el temario**.

Índice

§	Tema	Unidad	Prioridad
D1	Hoy: día 1 (lunes 9), horario y primera tarea	U01 y U02	★
0	Plan de estudio previo (fin de semana)	—	○
1	El programa en una página	Todas	★
2	Checklist del entorno	—	★
3	IA generativa y SDLC	U01	★
4	Desarrollo agéntico y Spec-Driven Development	U02	★
5	Generación, refactorización, documentación y Agent Skills	U03	★
6	Sistemas legados: diagnóstico y modernización	U04 y U05	★
7	Calidad, pruebas y CI/CD	U06	★
8	Code review y colaboración	U07	★
9	Gobernanza, seguridad y uso responsable	U08	★
10	Madurez, adopción, plugins y MCP	U09	★
11	ROI, productividad y taller integrador	U10	★
12	Unidad por unidad: qué ya tenemos en Rayen y qué nos falta	Todas	★
13	Contexto previo: Simplifique y la Plataforma IA de Rayen	U09 y U10	○
14	Siglas	—	★
15	Glosario	—	★
16	Portugués de bolsillo	—	○
17	Tarjetas de repaso	—	★
18	Llegar con ventaja: borradores de los primeros entregables	U01 y U09	★
19	Preguntas para el instructor	—	★
20	Fuentes	—	○

Hoy: día 1 (lunes 9 de noviembre) ★

Hoy se ven las dos primeras unidades: **U01** en la mañana (IA generativa y SDLC) y **U02** en la tarde (desarrollo agéntico y SDD). La meta del día es terminar con el entorno funcionando, el mapa de uso de IA versionado y una primera feature hecha a partir de una spec.

Horario

Hora	Qué pasa	Qué llevar en la cabeza
08:45	Llegar con Claude Code funcionando (en clase no se instala desde cero)	§D1 «Antes de entrar»
09:00 – 10:25	**U01 cátedra:** presentación, LLM, asistente frente a agente, vibe coding frente a SDD, marco 4D	§3
10:25 – 10:40	Receso	—
10:40 – 12:00	**Taller U01 «Mi entorno y mi mapa»** (la primera tarea)	Tabla de abajo
12:00 – 13:00	Almuerzo	—
13:00 – 14:25	**U02 cátedra:** ciclo del agente, contexto, spec primero, «tú firmas»	§4
14:25 – 14:40	Receso	—
14:40 – 16:00	**Taller U02 «Mi primera feature guiada por spec»** (individual)	Tabla de abajo

Antes de entrar: cinco minutos que evitan perder la mañana

```
claude --version        # debe ser 2.1.277 o superior; si no:  claude update
dotnet --list-sdks      # debe aparecer un SDK 10.x
git --version
ls ~/CLAUDE.md ~/.claude/CLAUDE.md   # si alguno existe, renómbralo durante el curso
```

> **Regla clave del curso: `CLAUDE.md` apaga `AGENTS.md`.** El curso trabaja con `AGENTS.md`, pero Claude Code solo lo lee si **no existe ningún `CLAUDE.md`**, ni en el proyecto ni en carpetas superiores (incluido el home). Por eso: los repos del curso van en una carpeta aparte (por ejemplo `~/curso-vela/`), nunca dentro de una carpeta de Rayen; no se usa `/init` en esos repos (genera un `CLAUDE.md`); y en `/config` se activa la lectura de instrucciones del proyecto.

La primera tarea: taller U01 «Mi entorno y mi mapa» (80 min)

El caso: entras al equipo que heredó «Pedidos Legado → Pedidos Web», el sistema de pedidos de una distribuidora hecho en Delphi con DBISAM. Hoy cada uno usa la IA a su manera y nadie sabe en qué etapas ayuda de verdad. El líder técnico pide dos cosas: un entorno igual para todos y un mapa de dónde aporta la IA y con qué control.

Paso	Min	Qué haces	Hecho
1	35	Autenticar Claude Code con la cuenta Team y verificar la versión (2.1.277 o más). Activar en `/config` la lectura de instrucciones del proyecto y confirmar que no hay `CLAUDE.md` en el home ni en carpetas superiores. Activar la licencia JetBrains, instalar Rider y WebStorm, instalar el plugin **Claude Code [Beta]** y abrir `claude` desde la terminal del IDE. Crear un **Project** en Claude web con el README del caso. **Punto de control:** al preguntarle por las reglas del proyecto, Claude Code cita el `AGENTS.md` de ejemplo	☐
2	8	Responder el cuestionario de madurez y pedirle a Claude que tabule las respuestas	☐
3	12	Implementar la función `formatearFolio()` del README, **primero sin IA y después con Claude Code**. Anotar tiempo, errores y qué hubo que corregir	☐
4	20	Construir `docs/mapa-sdlc.md`: por cada etapa, qué se delega a Claude, qué artefacto queda versionado, qué revisa una persona, qué riesgo hay y en qué evento de Scrum ocurre (base: borrador §18.1)	☐
5	5	Commit del mapa y cierre grupal	☐

Lo que se espera: el mapa no dice «la IA hace todo»; en cada etapa queda claro dónde decide una persona. Si en el paso 3 la IA cometió un error que tú corregiste, úsalo como evidencia en el mapa.

Si algo falla: si falla la terminal, usar la pestaña «Code» de Claude Desktop; si falla el plugin, correr `claude` en una terminal externa y conectarlo con `/ide`; si falla la red, el mapa se hace en papel con la plantilla impresa.

Demos que verás en la cátedra: el mismo prompt dos veces (una función que valida un RUT chileno, atentos al dígito verificador «K»); el mismo pedido en chat, en un Project y en Claude Code sobre la app ficticia ReservaCancha; y la actividad «¿Delegar o no?» con 8 tarjetas, decidiendo qué D pesa más en cada una.

La tarea de la tarde: taller U02 «Mi primera feature guiada por spec» (80 min, individual)

El caso: primer sprint del portal web de Pedidos, una API en ASP.NET Core con datos en memoria. La historia del backlog es vaga a propósito: «Como vendedor, quiero listar pedidos filtrando por cliente, estado y rango de fechas». No dice qué estados existen, qué pasa si las fechas vienen invertidas ni cuántos resultados se devuelven.

Paso	Min	Qué haces	Hecho
1	10	Ejecutar el **prompt maestro**: Claude te entrevista y genera borradores de `AGENTS.md` y `spec.md` con secciones `TODO`	☐
2	15	Completar la spec **tú mismo**: estados, validación de fechas, paginación y errores. **Al menos 5 criterios de aceptación verificables** (el instructor revisa uno)	☐
3	15	Claude Code en **plan mode**: pedir el plan, cuestionarlo, aprobarlo y guardarlo como `plan.md`	☐
4	25	Implementar con un test xUnit por criterio y correr `dotnet test`	☐

Paso	Min	Qué haces	Hecho
5	10	Revisar el diff con el checklist de §4.5, corregir y hacer commit	☐
6	5	Cierre: algunos muestran un criterio y el test que lo prueba	☐

Lo que se espera: el prompt maestro te ahorra el formato, no las decisiones. Una buena spec permite que otra persona escriba los tests sin preguntarte nada. Si Claude implementa algo que no está en la spec, es un hallazgo de revisión, no un regalo. Si te atrasas en el paso 2, existe la rama `u2-respaldo` con la spec ya escrita.

Demos y actividad de la cátedra: plan mode en vivo sobre la app ficticia TodoMini; el mismo pedido con y sin `AGENTS.md` mirando `/context`; «prompt suelto frente a spec» en TiendaMini; y «Caza el error en el diff», un diff con tres problemas plantados: un caso borde sin cubrir, una API key en el código y un test que siempre pasa.

Para hacer en casa esta noche (si queda energía)

Repasar las tarjetas 13 a 23 (§17) y leer §5 (U03, mañana martes en la mañana).

0. Plan de estudio previo (fin de semana) ○

Método: en cada bloque, primero lee (la mitad del tiempo), después explica el tema en voz alta sin mirar y al final repasa las tarjetas del bloque (§17). Si no puedes explicar algo en un minuto, todavía no lo sabes.

Sábado 7: días 1 a 3 del curso (unas 4 horas)

Bloque	Duración	Leer	Al terminar debes poder…
1	30 min	§1 Programa y §2 Checklist	Decir de memoria las 10 unidades y la cadena de entregables
2	45 min	§3 (U01)	Explicar el marco 4D, las superficies de Claude y la diferencia entre asistente y agente
3	50 min	§4 (U02)	Escribir de memoria la cadena `intent.md → spec.md → plan.md → tasks.md` y decir qué va en cada archivo
Pausa	15 min	—	—
4	40 min	§5 (U03)	Explicar cuándo usar una Skill y cuándo AGENTS.md; armar el frontmatter de un SKILL.md
5	60 min	§6 (U04 y U05)	Explicar strangler fig, golden master y la matriz «mantener, refactorizar, modernizar o reemplazar»

Domingo 8: días 4 y 5 del curso, y repaso (unas 4 horas)

Bloque	Duración	Leer	Al terminar debes poder…
6	40 min	§7 (U06) y §8 (U07)	Explicar las cuatro reglas de un pipeline seguro y qué hace `claude-code-action`
7	40 min	§9 (U08)	Explicar la jerarquía de settings, allow/deny, sandbox y la «política ejecutable»
8	40 min	§10 (U09) y §11 (U10)	Explicar plugins y marketplaces, MCP, las métricas DORA y un ROI con escenarios
Pausa	15 min	—	—
9	30 min	§12 Rayen y §14 Siglas	Saber dónde Rayen ya va adelantada y dónde no
10	45 min	§17 Tarjetas (todas) y §18 Borradores	Pasar las tarjetas; terminar el borrador del mapa de uso de IA
11	15 min	§19 Preguntas	Elegir qué preguntas hacer primero

1. El programa en una página ★

1.1 Las diez unidades

Unidad	Fecha	Tema	Conceptos clave	Entregable
U01	Lun 9, mañana	IA generativa aplicada al desarrollo y al SDLC	Tokens, contexto, alucinaciones; asistente frente a agente; superficies y modelos; vibe coding frente a ingeniería asistida; **marco 4D**; **AGENTS.md**	Mapa de uso de IA en el ciclo de desarrollo del equipo
U02	Lun 9, tarde	Desarrollo agéntico y Spec-Driven Development	Ciclo reunir contexto → actuar → verificar; modos de permiso; **plan mode**; `/context`, `/compact`, `/clear`; **intent → spec → plan → tasks**; «tú firmas»	Flujo completo guiado por spec (AGENTS.md, spec, plan, código y tests)
U03	Mar 10, mañana	Generación, refactorización y documentación	Pasos chicos; red de tests; **rewind** y git; **Agent Skills** (SKILL.md); desconfía y verifica	Módulo funcional documentado con IA
U04	Mar 10, tarde	Análisis y diagnóstico de sistemas legados	Plan mode de solo lectura; **subagentes** de lectura; diagramas **Mermaid**; reglas de negocio con archivo y línea; matriz ponderada	Diagnóstico técnico del sistema legado
U05	Mié 11, mañana	Modernización y migración	**Strangler fig**; tests de caracterización (**golden master**); equivalencias Delphi → C# y DBISAM → PostgreSQL; hooks y subagentes por rol	Plan de modernización, con el módulo migrado a C# y sus tests
U06	Mié 11, tarde	Calidad, pruebas y CI/CD	Pirámide de tests; bug → test que falla → corrección; cobertura ≠ calidad; pipeline seguro; **claude-code-action**; modo **headless**	Pipeline con tests asistidos por IA
U07	Jue 12, mañana	Code review y colaboración	La IA comenta, tú decides; PR chico; **REVIEW.md**; carpeta `.claude/` versionada; `/code-review`; subagente revisor	Proceso de code review asistido por IA
U08	Jue 12, tarde	Gobernanza, seguridad y uso responsable	Todo input es sospechoso; **prompt injection**; mínimo privilegio; allow/deny; **sandbox**; datos del plan Team; **política ejecutable** (settings, hooks, configuración administrada, plugin security-guidance)	Guía de gobernanza para el uso de IA
U09	Vie 13, mañana	Madurez organizacional y adopción	**Modelo de madurez de 5 niveles**; piloto y *champions*; **playbook como código** (plugins y marketplace); **MCP**; portabilidad	Plan de evolución de la IA para una Business Unit, con playbook instalable
U10	Vie 13, tarde	ROI, productividad y taller integrador	**DORA** y métricas de adopción; **ROI con supuestos**, escenarios y sensibilidad; informe ejecutivo de una página	Plan integrado de adopción de la IA

1.2 La cadena de entregables

Cada entregable alimenta al siguiente; el de la U10 integra los nueve anteriores:

```
U01 Mapa de uso de IA → U02 Flujo por spec → U03 Módulo documentado → U04 Diagnóstico del legado
→ U05 Plan de modernización → U06 Pipeline con tests → U07 Proceso de code review
→ U08 Guía de gobernanza → U09 Plan de evolución de la BU → U10 Plan integrado de adopción
```

Consecuencia práctica: no hay que quedarse atrás el día 1, porque todo se acumula. Las U06 y U07 se trabajan en parejas y las U09 y U10 en tríos.

1.3 Las seis etapas del SDLC según el programa

Requisitos → Pruebas → Validación → Revisión → Evolución → Mantención. Claude y Claude Code se integran en cada una, sobre tres estándares abiertos: **MCP**, **Agent Skills** y **AGENTS.md**.

Ojo: no son las mismas seis etapas de la pantalla de Simplifique (§13). El programa pone el acento en la evolución y la mantención (legado), no en el despliegue.

1.4 Las 11 competencias, en una línea cada una

1. Claude y Claude Code en todo el SDLC.
2. SDD con agentes, del requisito al commit revisado.
3. Contexto con AGENTS.md, Agent Skills y subagentes.
4. Tests significativos y CI/CD con GitHub Actions.
5. IA en code review, documentación y colaboración.
6. Evaluar sistemas legados.
7. Planificar refactorización y migración incremental (Delphi → .NET, DBISAM → PostgreSQL).
8. Gobernanza y seguridad **como configuración verificable**.
9. Madurez organizacional y buenas prácticas empaquetadas en plugins y playbooks.
10. Medir productividad, calidad y ROI.
11. Elegir superficie y modelo por tarea, sobre estándares abiertos.

1.5 Ideas fuerza del programa

- **«Tú firmas»:** la persona responde por cada commit, aunque lo haya escrito la IA.
- **«La IA propone, la persona decide»** (CI) y **«La IA comenta, tú decides»** (code review).
- **«Desconfía y verifica.»**
- **«Pasos chicos: una tarea, un incremento, un commit.»**
- **«Todo input es sospechoso.»**
- **«Política ejecutable»:** una regla que no está en la configuración es solo un deseo.
- **Portabilidad:** lo aprendido sobre MCP, Agent Skills y AGENTS.md sirve aunque se cambie de herramienta.

2. Checklist del entorno ★

#	Tarea	Para cuándo	✔
1	Claude Code instalado y autenticado con la cuenta Team, versión **2.1.277 o superior** (`claude --version`; actualizar con `claude update`)	Lunes 9	☐
2	En `/config`, activar la lectura de instrucciones del proyecto	Lunes 9	☐
3	Sin `CLAUDE.md` en el home (`~/CLAUDE.md`, `~/.claude/CLAUDE.md`) ni en carpetas superiores a los repos del curso; repos del curso en una carpeta aparte (`~/curso-vela/`)	Lunes 9	☐
4	Licencia JetBrains activada (invitación por correo, unos 10 meses); Rider y WebStorm instalados con el plugin **Claude Code [Beta]**. VS Code también sirve, pero el curso recomienda Rider	Lunes 9	☐
5	SDK de **.NET 10** instalado desde dotnet.microsoft.com/download (`dotnet --list-sdks`)	Lunes 9	☐
6	Git instalado y configurado	Lunes 9	☐
7	Haber recorrido el sitio del curso (ffelipecuevasc.github.io/curso-ia-desarrolladores)	Lunes 9	☐
8	**Docker** funcionando (`docker --version`); en la U05 se levanta PostgreSQL 16 con un solo comando	Miércoles 11	☐
9	**Cuenta de GitHub** para GitHub Actions (U06 y U07). La credencial de Claude para Actions la gestiona el instructor	Miércoles 11	☐
10	Si no hay cuenta Team, el material trae un plan B con Claude Pro, pero la política de Vela aprueba Claude **solo en versiones empresariales**: confirmarlo antes con el OSI	Antes de usar Pro	☐
11	Confirmar con el OSI que en el curso solo se usará el repositorio sintético (ver advertencia 2)	Lunes 9	☐
12	Borrador del mapa de uso de IA (§18.1) a mano	Lunes 9	☐

3. U01: IA generativa aplicada al desarrollo y al SDLC ★

3.1 Los LLM en la práctica

Concepto	Qué es	Consecuencia práctica
Token	Fragmento de texto que procesa el modelo	Mide costo y contexto. El español gasta más que el inglés
Ventana de contexto	Todo lo que el modelo ve a la vez	Cuando se llena, la calidad cae. Conviene una conversación por tarea
Variabilidad	La misma pregunta da respuestas distintas	Por eso hacen falta specs, tests y revisión
Alucinación	Respuesta plausible pero falsa (una API, un paquete o una regla inventados)	Verificar siempre, sobre todo dependencias y APIs

3.2 Asistente y agente; superficies y modelos

	Asistente	Agente
Qué hace	Responde y sugiere	Recibe un objetivo, usa herramientas (archivos, terminal) y avanza en varios pasos
Quién ejecuta	La persona	El agente, con permisos y aprobaciones
Ejemplo	Claude en la web, Projects	Claude Code

Superficies de Claude que menciona el programa:

Superficie	Para qué sirve
Web (claude.ai)	Conversar, analizar, redactar
Projects	Conversaciones con documentos y contexto compartido por un equipo
Desktop	Aplicación de escritorio
Claude Code	Agente de desarrollo en la terminal
Plugin JetBrains	Claude Code integrado al IDE (Rider, WebStorm)

Modelos:

Modelo	Perfil	Cuándo usarlo
Haiku	Rápido y económico	Tareas mecánicas: resumir, clasificar, generar datos de prueba
Sonnet	Equilibrado	El día a día: implementar, refactorizar
Opus	El más capaz	Diseño, planes, revisión crítica, diagnóstico de legado

3.3 Del vibe coding a la ingeniería asistida

	Vibe coding	Ingeniería asistida (SDD)
Cómo se trabaja	Pedir y aceptar lo que salga	Spec, plan, incrementos, tests y revisión

	Vibe coding	Ingeniería asistida (SDD)
Sirve para	Prototipos desechables	Todo lo que tendrá usuarios, datos o mantenimiento
Referencia	Addy Osmani, *Beyond Vibe Coding* (O'Reilly, 2025)	—

Scrum y SDD conviven: la historia de usuario del backlog se convierte en spec durante el refinamiento; los criterios de aceptación de la spec forman parte de la definición de terminado (*Definition of Done*); el sprint se ejecuta por tareas (`tasks.md`).

3.4 El marco 4D (AI Fluency, Anthropic) ★

Cuatro competencias para trabajar con IA de forma responsable:

D	Pregunta	En desarrollo
Delegación	¿Qué hago yo, qué hace la IA y con qué herramienta?	Decidir qué tareas delegar al agente y en qué nivel de autonomía
Descripción	¿Cómo comunico bien lo que necesito?	Specs, prompts con objetivo y criterios, AGENTS.md
Discernimiento	¿Cómo evalúo lo que produjo?	Leer el diff, correr tests, detectar alucinaciones
Diligencia	¿Cómo me hago responsable del resultado?	«Tú firmas», transparencia sobre el uso de IA, seguridad, no subir datos sensibles

Truco para recordarlo: **antes** (Delegación y Descripción), **después** (Discernimiento) y **siempre** (Diligencia).

3.5 AGENTS.md: la convención del curso ★

- Archivo Markdown en la raíz del repositorio con las instrucciones del proyecto para cualquier agente: cómo compilar, probar, convenciones y prohibiciones.
- Es un **estándar abierto** (agents.md), administrado junto con MCP en la Agentic AI Foundation de la Linux Foundation (aaif.io).
- Puede haber archivos AGENTS.md anidados en subcarpetas; se aplica el más cercano al archivo que se edita.
- Claude Code tiene su propio archivo, **CLAUDE.md**. Desde la versión **2.1.277** acepta `AGENTS.md`, pero **solo como respaldo**: lo lee únicamente si no existe ningún `CLAUDE.md`, ni en el proyecto ni en carpetas superiores como el home. Hay que activar la lectura de instrucciones del proyecto en `/config`.
- `/init` genera un `CLAUDE.md`, y eso desactivaría el `AGENTS.md`: no usarlo en los repos del curso.

En Rayen: usamos `CLAUDE.md` con el Método FICHA (secciones F, I, C, H y A). Como `CLAUDE.md` apaga `AGENTS.md`, en el curso hay que trabajar fuera de las carpetas de Rayen. Pregunta para el instructor: ¿conviene un `CLAUDE.md` que importe un `AGENTS.md` común? (§19).

3.6 Taller U01: «Mi entorno y mi mapa»

Entorno verificado (Claude Code, Rider, plugin, Project con el README del caso); cuestionario de madurez tabulado por Claude; la función `formatearFolio()` hecha sin IA y con IA (tiempo, errores y correcciones); y `docs/mapa-sdlc.md` con los **puntos de decisión humana**. Paso a paso en §D1. → Base: borrador de §18.1.

Notas del curso

4. U02: Desarrollo agéntico y Spec-Driven Development ★

4.1 El ciclo del agente

```
reunir contexto → actuar → verificar → (repetir hasta cumplir)
```

Claude Code lee archivos, busca y entiende (contexto); edita y ejecuta comandos (acción); corre tests, build y lint (verificación). Si no tiene cómo verificar, no tiene cómo saber que terminó bien: **dale siempre un comando de verificación**.

4.2 Modos de permiso (se alternan con Shift+Tab)

Modo	Qué hace	Cuándo usarlo
Predeterminado	Pide aprobación para editar y para ejecutar	Por defecto
Aceptar ediciones	Edita sin preguntar; pide para comandos	Tareas acotadas con tests
Plan mode	Solo lee y propone un plan; no modifica nada	Entender, diseñar, legado (U04)
Bypass	Sin aprobaciones	**Nunca** en trabajo real (en Rayen está bloqueado)

4.3 Gestión del contexto

Comando	Qué hace	Cuándo
`/context`	Muestra cuánto contexto se usa y en qué	Cuando las respuestas empeoran
`/compact`	Resume la conversación para liberar espacio	En medio de una tarea larga
`/clear`	Empieza de cero	Al cambiar de tarea

AGENTS.md (o CLAUDE.md) es la **memoria del proyecto**: lo que el agente necesita saber en todas las sesiones. Lo puntual de una tarea va en la spec, no en AGENTS.md.

4.4 Spec primero: la cadena de archivos ★

```
intent.md  →  spec.md  →  plan.md  →  tasks.md  →  código + tests  →  revisión del diff  →  commit
 por qué      qué          cómo         en qué orden
```

Archivo	Contenido	Pregunta que responde
`intent.md`	Problema, para quién y por qué ahora; resultado esperado	¿Por qué hacemos esto?
`spec.md`	Comportamiento, alcance (qué incluye y qué no), **criterios de aceptación verificables**	¿Qué debe hacer, exactamente?
`plan.md`	Diseño técnico, archivos a tocar, decisiones y alternativas	¿Cómo lo construimos?
`tasks.md`	Lista ordenada de incrementos pequeños, cada uno verificable	¿En qué orden y en qué pasos?

Un criterio de aceptación verificable se puede convertir directamente en un test. Formato útil (Dado / Cuando / Entonces):

> Dado un pedido con 3 ítems de $1.000 y un descuento del 10 %, cuando se calcula el total, entonces el total es $2.700.

En el laboratorio: **un test xUnit por criterio**.

Herramientas parecidas que pueden aparecer en el curso: GitHub Spec Kit (*constitution* → *specify* → *plan* → *tasks*) y Kiro de AWS (*requirements* → *design* → *tasks*). En Rayen, SGI-PR-040 usa `spec.md → plan.md → tasks.md`.

4.5 «Tú firmas»: checklist de revisión del diff

Antes de cada commit:

1. ¿Leí el diff **completo**?
2. ¿Cada cambio responde a un criterio de la spec, o hay cosas que no pedí?
3. ¿Los tests prueban el criterio, o solo pasan?
4. ¿Hay dependencias nuevas? ¿Existen y son las correctas?
5. ¿Hay secretos, datos reales o logs con información sensible?
6. ¿Puedo explicar cada línea?
7. ¿El mensaje del commit cuenta qué y por qué?

4.6 Taller U02: «Mi primera feature guiada por spec»

Individual. La historia «listar pedidos filtrando por cliente, estado y rango de fechas» se convierte en spec con al menos 5 criterios (con ayuda de un prompt maestro); el plan se revisa y aprueba en plan mode y se guarda como `plan.md`; se implementa con un test xUnit por criterio; se revisa el diff y se hace commit. Paso a paso en §D1.

Notas del curso

5. U03: Generación, refactorización, documentación y Agent Skills ★

5.1 Pasos chicos y red de seguridad

- **Una tarea, un incremento, un commit.** Si el incremento falla, se pierde poco.
- **Refactorizar** es cambiar la estructura **sin** cambiar el comportamiento. Los tests en verde antes y después son la prueba.
- **Rewind:** Claude Code guarda puntos de control y permite volver atrás la conversación y los archivos (Esc dos veces o `/rewind`). **Git** es la segunda red: commits frecuentes y ramas.

5.2 Agent Skills ★

Una **Skill** es una carpeta con un `SKILL.md` (y, si hace falta, scripts y plantillas) que enseña al agente a hacer una tarea específica. Es un **estándar abierto** (agentskills.io).

```
---
name: documentar-endpoint
description: Documenta un endpoint .NET con el formato del equipo. Úsala cuando se pida documentar una
API o un endpoint.
---

# Documentar un endpoint

1. Lee el endpoint y su validador.
2. Completa la plantilla `plantilla.md` con: propósito, request, response, errores y ejemplo.
3. Verifica que cada código de error del código aparezca en la documentación.
```

Carga progresiva: al inicio el agente solo ve el `name` y la `description` de cada Skill. Carga el cuerpo cuando la tarea coincide, y los archivos de apoyo cuando los necesita. Por eso la `description` debe decir **cuándo** usarla.

	AGENTS.md	Skill
Se carga	Siempre, en cada sesión	Solo cuando la tarea lo requiere
Contiene	Reglas generales del proyecto, comandos, convenciones	Un procedimiento específico, con plantillas o scripts
Ejemplo	«Los tests usan xUnit; nunca `SELECT *`»	«Cómo generar la documentación de un módulo»
Riesgo si se abusa	Contexto inflado en todas las sesiones	Skills que nunca se activan porque su descripción es vaga

5.3 Desconfía y verifica: prompt vago y prompt con criterios

Prompt vago	Prompt con objetivo y criterios
«Mejora el módulo de pedidos»	«Refactoriza `CalcularTotal` en `Pedidos.cs` para separar el cálculo de descuentos en un método propio. Criterios: los tests actuales siguen en verde; no cambia la firma pública; agrega un test para descuento 0 % y uno para descuento 100 %. No toques otros archivos.»

Partes de un buen prompt: **objetivo, contexto (archivos), restricciones, criterios de éxito y formato**.

5.4 Taller U03: «Módulo funcional y documentado»

Una funcionalidad en al menos tres incrementos; refactorización de un módulo heredado con tests en verde; una Skill que estandariza la documentación; comparación entre un prompt vago y uno con criterios.

Notas del curso

6. U04 y U05: sistemas legados, del diagnóstico a la migración ★

6.1 Primer de Delphi y DBISAM (para quien no los conoce)

Término	Qué es
Delphi	Entorno de desarrollo (IDE y compilador) de Embarcadero para Object Pascal; muy usado en aplicaciones de escritorio de gestión
Object Pascal	Lenguaje de Delphi
Unit (`.pas`)	Archivo de código: tiene una sección `interface` (pública) y una `implementation`
Formulario (`.dfm`)	Definición de una pantalla y sus componentes; suele tener lógica de negocio pegada a los eventos (`OnClick`)
Proyecto (`.dpr`)	Programa principal
DataModule	Unit que agrupa componentes de datos (tablas, queries)
DBISAM	Motor de base de datos embebido para Delphi (Elevate Software), basado en archivos; sin servidor
VCL	Biblioteca de componentes visuales de Delphi

Por qué cuesta migrarlo: las reglas de negocio están escondidas en eventos de formularios, en triggers o en código de DataModules, y la base es local, sin las restricciones de una base relacional moderna.

6.2 U04: diagnóstico de solo lectura

Técnica	Cómo se aplica
Mirar sin tocar	Plan mode; preguntas de comprensión («¿dónde se calcula el total?»); diagramas **Mermaid** de dependencias y flujos
Divide y explora	**Subagentes de lectura**: cada uno explora una parte con su propio contexto y devuelve solo la conclusión. Así el contexto principal no se llena
Reglas ocultas	Extraer cada regla de negocio **con archivo y línea**, y verificarla en el código; nunca aceptar una regla sin su referencia
Decidir con datos	Matriz ponderada por módulo: mantener, refactorizar, modernizar o reemplazar

Matriz de decisión ponderada (ejemplo ilustrativo):

Criterio (peso)	Módulo Pedidos	Módulo Clientes
Valor para el negocio (30 %)	5	3
Deuda técnica (25 %)	4	2
Riesgo de cambio (20 %)	3	2
Cobertura de tests (15 %)	1	1
Dependencias (10 %)	4	2
Decisión	**Modernizar**	**Mantener**

Se relaciona con las «R» de modernización usadas en la industria: retener, retirar, rehostear, replataformar, refactorizar, rearquitecturar, reemplazar.

Diagrama Mermaid (el agente los genera y se ven en GitHub, Azure DevOps y VS Code):

```
graph LR
  FrmPedido --> DmPedidos
  DmPedidos --> TblPedidos[(Pedidos.dat)]
  FrmPedido --> UCalculos
```

Entregable U04: inventario de units, formularios y tablas; mapa de dependencias; catálogo de reglas de negocio; riesgos y deuda priorizados con evidencia; matriz por módulo y resumen ejecutivo para la gerencia.

6.3 U05: migración incremental

Strangler fig (higuera estranguladora, Martin Fowler): el sistema nuevo crece alrededor del viejo y lo reemplaza **por partes**. Una fachada (proxy, gateway o capa anticorrupción) decide qué va al sistema nuevo y qué al viejo. Cuando ya nada pasa por el viejo, se apaga.

```
          ┌──────────── fachada ────────────┐
pedido →  │  ¿ruta migrada?  sí → .NET + PostgreSQL
          │                  no → Delphi + DBISAM
          └─────────────────────────────────┘
```

Tests de caracterización o golden master (Michael Feathers, *Working Effectively with Legacy Code*): antes de migrar se **congela el comportamiento actual**. Se ejecuta el sistema viejo con muchas entradas, se guardan las salidas (el «golden master») y el código nuevo debe producir **exactamente** las mismas. Si el viejo tenía un bug, primero se documenta y después se decide si corregirlo.

Traducir no es copiar: equivalencias Delphi → C# (verificar los detalles en el curso)

Delphi	C#	Cuidado
`Integer`	`int`	—
`Int64`	`long`	—
`Double`	`double`	Para dinero, no
`Currency`	`decimal`	`Currency` es de punto fijo con 4 decimales
`string`	`string`	Los índices de string en Delphi empiezan en 1; en C#, en 0
`Boolean`	`bool`	—
`TDateTime`	`DateTime`	En Delphi es un `Double` (días desde 1899-12-30)
`TStringList`, `TList`	`List<T>`	—
`try … finally … Free`	`using` / `try … finally`	En C# la memoria la libera el recolector; `IDisposable` para recursos
`Round()`	`Math.Round()`	Ambos usan redondeo bancario por defecto (al par); verificar contra el golden master

Tipos DBISAM → PostgreSQL (verificar en el curso)

DBISAM	PostgreSQL	Cuidado
String(n)	`varchar(n)`	Codificación: el legado suele estar en ANSI/Latin-1; PostgreSQL usa UTF-8
Integer / SmallInt / LargeInt	`integer` / `smallint` / `bigint`	—
AutoInc	`identity`	**En Rayen los IDs son UUID v7, no autoincrementales**
Float	`double precision`	—
Currency / BCD	`numeric(p,s)`	Nunca `float` para montos
Boolean	`boolean`	—
Date / Time / TimeStamp	`date` / `time` / `timestamp`	Zona horaria: decidir `timestamp` frente a `timestamptz`
Memo	`text`	—
Blob / Graphic	`bytea`	—

Verificación automática: un **hook** que corre los tests después de cada edición y **subagentes por rol** (uno migra, otro revisa, otro compara contra el golden master).

Convivencia y criterio de corte: durante un tiempo, ambos sistemas funcionan a la vez. El **criterio de corte** es la condición medible para apagar el viejo (por ejemplo, 30 días con el golden master en verde y sin tráfico hacia la ruta vieja).

Taller U05 «Migrar el cálculo de totales»: tests xUnit parametrizados desde el golden master; conversión a C# con un hook de tests; corrección documentada de defectos; esquema PostgreSQL, carga de datos y validación de totales; plan con convivencia y criterio de corte.

En Rayen: es exactamente nuestra migración de **Rayen 2020 a .NET + PostgreSQL con strangler fig**. Por ejemplo, el mantenedor de laboratorio es híbrido (frontend nuevo sobre el backend y la base legada) y se muda a `Release/` cuando ninguna ruta toque la base legada (lo verifica la telemetría del ACL del gateway), con UUID v7, auditoría Outbox, OPA y sensor local en verde. Ese es nuestro **criterio de corte**.

Notas del curso

7. U06: calidad, pruebas y CI/CD ★

7.1 Tests con criterio

Pirámide de tests: muchos tests unitarios (rápidos y baratos), menos de integración y pocos de punta a punta (E2E, lentos y frágiles).

Bug → test que falla → corrección: ante un bug, primero se escribe un test que lo reproduce (en rojo), después se corrige (en verde). El bug no vuelve sin que alguien lo note.

Cobertura ≠ calidad: la cobertura dice qué líneas se ejecutaron, no si se verificó algo. Un test sin `Assert` cubre y no prueba nada. Señal mejor: **mutation testing** (Stryker.NET), que cambia el código a propósito y comprueba que algún test falle.

Casos que la IA suele olvidar y conviene pedir: valores límite (0, negativo, máximo), entradas inválidas o nulas, textos vacíos, fechas extremas, concurrencia.

7.2 Pipeline seguro

Regla	En GitHub Actions
Secretos fuera del código	`secrets.ANTHROPIC_API_KEY`, nunca en el YAML
Permisos mínimos	Bloque `permissions:` con solo lo necesario (por ejemplo, `contents: read`)
Versiones fijas	Acciones fijadas a una versión o, mejor, a un hash de commit; nada de `@latest`
Timeouts	`timeout-minutes` en cada job, para que un agente no corra sin límite

7.3 Claude en CI

Pieza	Qué es
claude-code-action	GitHub Action oficial de Anthropic (github.com/anthropics/claude-code-action). Corre Claude Code dentro del pipeline
@claude	Mencionar a Claude en un issue o PR para que responda, revise o proponga cambios
Modo headless	Claude Code sin conversación, desde un script: `claude -p "instrucción"`. Sirve en cualquier CI, no solo en GitHub
Regla	**La IA propone (comentario o PR), la persona decide (merge)**

Taller U06 (en parejas): tests de casos límite; workflow de build y tests en cada push y PR; Claude disponible vía @claude; una falla inyectada que se diagnostica y se corrige con revisión humana.

En Rayen: usamos **Azure Pipelines**, no GitHub Actions; `claude-code-action` no aplica directo. El equivalente sería un paso con `claude -p` en modo headless. Además, la etapa `ci_cumplimiento` de `Release/azure-pipelines.yml` todavía no se ejecutó nunca. Usamos **TUnit**, no xUnit (pregunta en §19).

Notas del curso

8. U07: code review y colaboración ★

Idea	Qué significa
La IA comenta, tú decides	Cada hallazgo de la IA se **acepta, se discute o se descarta con un argumento**. Ninguno se ignora en silencio
PR chico	Un tamaño que una persona puede revisar en serio; la descripción se genera desde el diff y se valida
AGENTS.md y REVIEW.md	AGENTS.md tiene las reglas para todo el trabajo. REVIEW.md tiene los criterios específicos de revisión (qué mirar, qué bloquea un merge)
Configuración compartida	La carpeta `.claude/` **versionada** en el repositorio: settings, comandos, subagentes y hooks iguales para todo el equipo
`/code-review`	Comando de revisión de Claude Code sobre los cambios
Subagente revisor	Agente de **solo lectura** con contexto limpio: no vio cómo se escribió el código, así que lo revisa como un tercero
GitHub Action de revisión	Revisión automática en cada PR

Taller U07 (parejas cruzadas): descripción de PR generada y validada; REVIEW.md y subagente revisor de solo lectura; workflow de revisión automática; revisión cruzada con respuesta a cada hallazgo y **al menos un hallazgo humano** (lo que la IA no vio).

En Rayen: ya tenemos el subagente `revisor` de solo lectura (`.claude/agents/revisor.md`), obligatorio en cambios clínicos y de autenticación, y la plantilla de PR en `.azuredevops/`. Nos falta un REVIEW.md explícito.

Notas del curso

9. U08: gobernanza, seguridad y uso responsable ★

9.1 Todo input es sospechoso

- **Código generado:** puede traer vulnerabilidades (inyección SQL, validaciones ausentes), dependencias inventadas o licencias incompatibles.
- **Prompt injection:** instrucciones maliciosas escondidas en algo que el agente lee (un README, un comentario, un issue o una página web). El agente puede obedecerlas como si vinieran de la persona.
- **Trifecta letal:** si un agente tiene acceso a datos privados, lee contenido no confiable y puede comunicarse hacia afuera, puede filtrar datos. Hay que romper al menos una de las tres.

9.2 Mínimo privilegio y defensa en profundidad

Capa	Qué hace
Reglas `allow` / `ask` / `deny`	Qué herramientas y comandos puede usar el agente (por ejemplo, `deny` para leer `.env` o para `git push --force`)
Sandbox	Ejecuta los comandos del agente aislados del sistema de archivos y de la red
Hooks	Scripts que revisan cada acción y pueden bloquearla
Revisión humana	Última capa

Ninguna capa basta sola: eso es **defensa en profundidad**.

9.3 Política ejecutable ★

Pieza	Qué es
Settings versionados	`.claude/settings.json` en el repositorio, igual para todos
Jerarquía de settings	Configuración administrada (de la organización) > línea de comandos > `settings.local.json` del proyecto > `settings.json` del proyecto > configuración del usuario. La administrada no se puede saltar
Hooks	`PreToolUse` (antes de una acción, puede bloquear), `PostToolUse` (después), `UserPromptSubmit` (al enviar un prompt), `Stop` (al cerrar el turno), `SessionStart`, entre otros
Configuración administrada	`managed-settings` que impone la organización (por ejemplo, desactivar el modo bypass)
Plugin security-guidance	Plugin de Anthropic que advierte sobre patrones inseguros mientras el agente edita

«Probadas contra violaciones»: una política solo cuenta si se intentó violarla y el control la detuvo.

9.4 Datos y propiedad intelectual

- **Términos del plan Team:** son términos comerciales. Repasar en el curso qué dicen sobre retención de datos y uso para entrenamiento, y compararlos con lo que exige la política de Vela (§13.4).
- **Protección de datos:** minimización, nada de datos personales reales, datos sintéticos.
- **Propiedad intelectual:** quién es dueño del código generado; licencias del código sugerido.

9.5 Evaluación de escenarios de riesgo

Escenario	Probabilidad	Impacto	Control
El agente lee un `.env` y lo muestra	Media	Alto	`deny` de lectura + hook de secretos
Un comentario del legado contiene una inyección de prompt	Baja	Alto	Plan mode, sin red, revisión humana
El agente agrega un paquete inexistente	Media	Medio	Revisión de dependencias + SCA en CI

Taller U08 (en parejas): escenarios de riesgo; auditoría cruzada del legado; políticas convertidas en configuración y probadas contra violaciones; directrices de uso para el equipo.

En Rayen: es nuestra unidad más fuerte. Tenemos el modo bypass desactivado; hooks de prompt, de archivos, de bash, de auditoría y de cierre de turno; `deny` para secretos y producción; hook de git `pre-commit` contra secretos y RUN; declaración de IA en cada commit; y cada control **se probó haciéndolo fallar** (2026-11-04). Ver la tabla «C · Controles» de `CLAUDE.md` y `.agents/harness/matriz-cumplimiento.md`. Se puede compartir **a nivel de principios**, sin rutas ni nombres internos.

Notas del curso

10. U09: madurez organizacional y adopción ★

10.1 Modelo de madurez de 5 niveles

El programa no detalla los niveles; hay que anotarlos en el curso. Un esquema típico, para tener una referencia:

Nivel	Típicamente	Señal
1	Uso individual y ad hoc	Cada quien usa lo que quiere
2	Uso guiado	Herramienta aprobada, primeras reglas
3	Estandarizado	AGENTS.md, Skills y settings compartidos por equipo
4	Medido	Métricas de adopción y DORA; controles en CI
5	Optimizado	Playbooks instalables, mejora continua con datos

10.2 Multiplicadores

Piloto (un equipo prueba antes que todos), **champions** (personas que contagian la práctica y ayudan a otras) y **el desarrollador senior como multiplicador** (revisa, enseña y escribe las Skills).

10.3 Playbook como código: plugins y marketplace ★

- Un **plugin** de Claude Code empaqueta **Skills, comandos, subagentes, hooks y servidores MCP** en una unidad instalable.
- Un **marketplace** es un catálogo de plugins (puede ser interno, en un repositorio Git). Se agrega con `/plugin marketplace add` y se instala con `/plugin install`.
- Resultado: el «playbook» del equipo deja de ser un documento y pasa a ser algo que se instala, se versiona y se actualiza.

10.4 MCP

- **Model Context Protocol:** estándar abierto para conectar agentes con sistemas externos.
- Arquitectura cliente-servidor. El **servidor MCP** expone **herramientas** (acciones), **recursos** (datos) y **prompts**. Se conecta por stdio (local) o por HTTP (remoto).
- En el laboratorio: un plugin con acceso **de solo lectura** a la base de datos vía MCP.
- Riesgo: cada servidor MCP es un proveedor más y una puerta para la inyección de prompts. Hay que aprobarlo antes de usarlo.

10.5 Herramienta justa y portabilidad

Elegir la **superficie** (web, Projects, Claude Code, IDE, CI) y el **modelo** (Haiku, Sonnet, Opus) según la tarea. Los tres estándares abiertos (MCP, Agent Skills y AGENTS.md) hacen que lo construido sirva aunque se cambie de herramienta.

Taller U09 (en tríos): diagnóstico de madurez con datos de adopción; plugin de playbook con MCP de solo lectura; una Skill que genera PDF con identidad visual; roadmap a 6 y 12 meses.

En Rayen: la escalera de la RFC (§13.3) y el Setup de Simplifique son exactamente esto: distribuir el harness como producto instalable. Nuestro catálogo de skills y sensores (`CatalogQuery`) es un primer paso hacia un marketplace interno.

Notas del curso

11. U10: ROI, productividad y taller integrador ★

11.1 Medir lo que importa

Familia	Métricas
DORA	Frecuencia de despliegue; tiempo de entrega de cambios (*lead time*); tasa de fallas por cambio; tiempo de recuperación. Los informes recientes agregan la tasa de retrabajo
Adopción	Personas activas, sesiones, uso por equipo, Skills y plugins usados (Claude Code tiene un panel de Analytics en los planes de equipo)
Calidad	Defectos en producción, PR aprobados a la primera, retrabajo
No sirven solas	Líneas de código generadas; número de prompts

11.2 ROI con supuestos

```
ROI = (beneficios − costos) / costos
```

Componente	Incluye
Costos	Licencias, consumo, horas de formación, tiempo de configuración, tiempo de revisión extra
Beneficios	Horas ahorradas × costo por hora; menos defectos; menos retrabajo; menor tiempo de entrega

Ejemplo ilustrativo (números inventados):

Supuesto	Conservador	Optimista
Personas	10	10
Horas ahorradas por persona al mes	4	12
Costo por hora	20	20
Beneficio mensual	800	2.400
Costo mensual (licencias, consumo, revisión)	600	600
ROI mensual	**33 %**	**300 %**

Sensibilidad: ¿qué supuesto mueve más el resultado? (Acá, las horas ahorradas.) Ese es el que hay que medir mejor.

Informe ejecutivo de una página: decisión que se pide, resultado esperado, supuestos explícitos, escenarios, riesgos e indicadores para seguir.

Taller integrador: datos reales del caso; modelo de ROI; informe ejecutivo y plan integrado en PDF; presentación y **cuestionario final**.

Notas del curso

12. Unidad por unidad: qué ya tenemos en Rayen y qué nos falta ★

Unidad	Ya tenemos	Diferencia o brecha
U01	Guía 01 de fundamentos; semáforo de información; SGI-PR-040	El **marco 4D** es nuevo para nosotros. Usamos CLAUDE.md (Método FICHA), y **CLAUDE.md apaga AGENTS.md**: en el curso hay que trabajar fuera de las carpetas de Rayen
U02	SDD en SGI-PR-040 (`spec.md`, `plan.md`, `tasks.md`); puerta de integración de tres preguntas	No usamos `intent.md`. Probamos con **TUnit**, no xUnit
U03	Skills propias (`backend:new-slice`, `frontend:new-module`, `opa:*`…)	Revisar que sus descripciones digan **cuándo** usarlas
U04 y U05	Migración real de **Rayen 2020 → .NET + PostgreSQL** con strangler fig; criterio de corte definido	¿Tenemos golden master de los cálculos legados? Hay que traducir los AutoInc a **UUID v7**
U06	Sensores centrales y locales; etapa `ci_cumplimiento` en Azure Pipelines	El curso usa GitHub Actions. Nuestra etapa **nunca se ejecutó**. Solo CatalogQuery tiene sensor local
U07	Subagente `revisor` de solo lectura; plantilla de PR	Falta un **REVIEW.md**
U08	Controles probados haciéndolos fallar; auditoría; modo bypass bloqueado	Revisar la configuración administrada (`managed-settings`) y el plugin security-guidance
U09	RFC de la Plataforma IA; catálogo de skills y sensores; niveles de autonomía (demo)	No tenemos plugins ni marketplace; EM1 sin persona asignada; ningún MCP aprobado
U10	Indicadores de SGI-PR-040	No medimos DORA ni adopción todavía

13. Contexto previo: Simplifique y la Plataforma IA de Rayen ○

Sirve para las U09 y U10 y para conversar con otras empresas de Vela.

13.1 La cadena

```
Vela (Constellation Software) → Contmatic (Brasil) → Simplifique Platform (plataforma interna de IA, más madura)
   → RFC «Plataforma Interna de IA de Rayen Salud» (agosto 2026)
   → respuesta de Rayen: .agents/plans/plataforma-ia/ y .agents/specs/plataforma-ia/
```

13.2 La pantalla SDLC de Simplifique: tres niveles de autonomía

Etapa	Nivel 1: Asistido	Nivel 2: Dirigido	Nivel 3: Delegado
Triage y specs	Humano lidera, IA asiste	IA lidera, aprobación humana	Bucle del agente (solo IA)
Revisión de specs	Humano lidera, IA asiste	IA lidera, aprobación humana	Bucle del agente (solo IA)
Desarrollo	Humano lidera, IA asiste	IA lidera, aprobación humana	Bucle del agente (solo IA)
QA	Humano lidera, IA asiste	IA lidera, aprobación humana	Bucle del agente (solo IA)
Revisión de PR	Solo humano	Humano lidera, IA asiste	Humano lidera, IA asiste
Despliegue	Solo humano	Solo humano	Humano lidera, IA asiste

Reconstruida en `modulo-sdlc`; no es documentación oficial. La revisión del PR y el despliegue nunca quedan en «solo IA». Para el nivel 3 hace falta un verificador automático. Este modelo encaja con el marco 4D: **Delegación** decide el nivel.

Otras pantallas de Simplifique: el **Cluster de Execução** (pedir que el agente de otra persona ejecute algo, con aceptación explícita) y el **Setup** (instalar el harness y las skills en cada máquina, que es un «playbook instalable»).

13.3 La respuesta de Rayen en cinco puntos

1. **Escalera de inversión:** primero el harness con controles efectivos en cada repositorio; luego EM1 (Platform Engineering), EM2 (catálogo compartido: real, solo con RayenEnRed) y EM3 (observabilidad: propuesta); el resto queda en backlog.
2. **Solo lectura:** la plataforma recibe resultados por push desde CI; no ejecuta nada en los repositorios.
3. **La VPN no es autenticación:** Keycloak se encarga del login y OPA de la autorización.
4. **Sensores con cuatro estados:** `OPEN`, `CLOSED`, `PARTIAL` y `SKIP`, sin falsos positivos.
5. **Stack ya en producción** y minimización de datos.

13.4 Política de IA de Vela (enero de 2026), lo esencial

Claude (M02) y Claude Code (D04) están aprobados, solo en versiones empresariales. No se permiten PII ni datos confidenciales sin autorización expresa. Exige un registro de uso por proyecto (herramienta, versión, proveedor, fecha, caso de uso y responsable) que se guarda 5 años. Quedan prohibidas DeepSeek, Grok y las herramientas de países sancionados. La política se revisa cada 6 meses. En Brasil

rige la **LGPD** (los datos de salud son sensibles; ANPD; RIPD para IA de alto riesgo). En Chile, las leyes **19.628 y 21.719** (esta última vigente desde el 1 de diciembre de 2026), **20.584** y **21.663**.

14. Siglas ★

Sigla	Desarrollo	En el curso
AAIF	*Agentic AI Foundation* (Linux Foundation)	Administra estándares abiertos como MCP y AGENTS.md
ACL	*Anti-Corruption Layer* (también *Access Control List*)	Capa que aísla el sistema nuevo del legado durante la migración
AGENTS.md	—	Archivo estándar de instrucciones del proyecto para agentes
API	*Application Programming Interface*	Interfaz entre programas
BU	*Business Unit*	Unidad de negocio (entregable de la U09)
CI/CD	*Continuous Integration / Continuous Delivery*	Pipeline automático de build, tests y despliegue
CLI	*Command Line Interface*	Claude Code es una CLI
DBISAM	Motor de base de datos embebido para Delphi	Base del sistema legado del caso
DDD	*Domain-Driven Design*	Diseño guiado por el dominio
DoD	*Definition of Done*	Definición de terminado en Scrum
DORA	*DevOps Research and Assessment*	Métricas de entrega de software
DPIA / RIPD	*Data Protection Impact Assessment* / Relatório de Impacto à Proteção de Dados	Evaluación de impacto en datos personales
E2E	*End to End*	Prueba de punta a punta
IDE	*Integrated Development Environment*	Rider, WebStorm, VS Code
IP	*Intellectual Property*	Propiedad intelectual (U08)
JSON / YAML	Formatos de datos y configuración	Settings de Claude Code (JSON); workflows de GitHub Actions (YAML)
KPI	*Key Performance Indicator*	Indicador clave
LGPD	Lei Geral de Proteção de Dados (Brasil)	Ley de datos de Brasil
LLM	*Large Language Model*	Modelo grande de lenguaje
MCP	*Model Context Protocol*	Estándar para conectar agentes con sistemas
MTTR	*Mean Time to Recovery*	Tiempo medio de recuperación
ORM	*Object-Relational Mapping*	Mapeo entre objetos y tablas (Entity Framework); Rayen usa Dapper
OWASP	*Open Worldwide Application Security Project*	Top 10 de riesgos de seguridad
PII	*Personally Identifiable Information*	Datos que identifican a una persona
PR	*Pull Request*	Solicitud de integrar cambios
QA	*Quality Assurance*	Aseguramiento de calidad
ROI	*Return on Investment*	Retorno de la inversión
SAST / DAST / SCA	Análisis estático / dinámico / de dependencias	Herramientas de seguridad en CI

Sigla	Desarrollo	En el curso
SDD	*Spec-Driven Development*	Desarrollo guiado por especificaciones
SDLC	*Software Development Life Cycle*	Ciclo de vida del software
SHA	*Secure Hash Algorithm*	Hash de commit con el que se fija la versión de una acción de CI
SKILL.md	—	Archivo principal de una Agent Skill
TDD	*Test-Driven Development*	Escribir el test antes del código
UTF-8 / ANSI	Codificaciones de caracteres	Problema típico al migrar el legado
UUID v7	Identificador único ordenable por tiempo	Convención de Rayen para los IDs
VCL	*Visual Component Library*	Componentes visuales de Delphi

15. Glosario ★

Término	Definición
Agent Skill	Carpeta con un SKILL.md que enseña al agente una tarea; se carga solo cuando hace falta
Agente	Sistema que recibe un objetivo y usa herramientas en varios pasos
Alucinación	Respuesta plausible pero falsa
Asistente	IA que responde o sugiere; la persona ejecuta
Carga progresiva	El agente ve primero solo el nombre y la descripción de cada Skill, y carga el resto cuando lo necesita
Champion	Persona que impulsa la adopción en su equipo
Cobertura	Porcentaje del código que ejecutan los tests; no mide si verifican algo
Configuración administrada	Settings impuestos por la organización, que el usuario no puede saltar
Convivencia	Período en que el sistema legado y el nuevo funcionan a la vez
Criterio de aceptación	Condición verificable que se convierte en un test
Criterio de corte	Condición medible para apagar el sistema legado
Defensa en profundidad	Varias capas de control independientes
Golden master	Salidas del sistema viejo guardadas como referencia; el nuevo debe reproducirlas
Headless	Claude Code sin conversación, desde un script (`claude -p`)
Hook	Script que corre en un evento del agente y puede bloquear la acción
Marco 4D	Delegación, Descripción, Discernimiento y Diligencia
Marketplace	Catálogo de plugins que se pueden instalar
Matriz ponderada	Tabla de criterios con pesos para decidir qué hacer con cada módulo
Mermaid	Lenguaje de texto para dibujar diagramas en Markdown
Mínimo privilegio	Dar solo los permisos indispensables
Mutation testing	Cambiar el código a propósito y comprobar que algún test falle
Plan mode	Modo en que el agente solo lee y propone; no modifica nada
Playbook como código	Buenas prácticas empaquetadas como plugin instalable
Plugin	Paquete instalable con Skills, comandos, subagentes, hooks y MCP
Política ejecutable	Una regla implementada como configuración o hook y probada contra violaciones
Prompt injection	Instrucciones maliciosas escondidas en contenido que lee el agente
Refactorizar	Cambiar la estructura del código sin cambiar su comportamiento
REVIEW.md	Archivo con los criterios de revisión de código del equipo
Rewind	Volver la conversación y los archivos a un punto anterior
Sandbox	Ejecución aislada del sistema de archivos y de la red
Sensibilidad	Análisis de cuánto cambia el resultado si cambia un supuesto

Término	Definición
Strangler fig	Migración incremental: el sistema nuevo reemplaza al viejo por partes, detrás de una fachada
Subagente	Agente auxiliar con contexto propio, que devuelve solo la conclusión
Superficie	Dónde se usa Claude: web, Projects, Desktop, Claude Code, IDE, CI
Test de caracterización	Test que documenta lo que el sistema hace hoy, sin juzgar si está bien
Unit	Archivo de código Delphi (`.pas`)
Vibe coding	Generar sin spec ni revisión; solo para prototipos desechables
Ventana de contexto	Todo lo que el modelo ve a la vez

16. Portugués de bolsillo ○

El programa está en español, pero en Vela LATAM puede haber colegas de Brasil (la ementa usa la palabra portuguesa «turma» para el grupo).

Portugués	Español
Rodar (os testes)	Ejecutar (las pruebas)
Subir pra produção	Pasar a producción
Esteira	Pipeline
Homologação	Ambiente previo a producción
Sustentação	Mantención y soporte
Demanda / chamado	Solicitud / ticket
Escopo / prazo	Alcance / plazo
Gargalo	Cuello de botella
Retrabalho / débito técnico	Retrabajo / deuda técnica
Legado	Sistema legado
Time	Equipo
Pronto	Listo, terminado
Esquisito	Raro (falso amigo)
Apagar / salvar	Borrar / guardar (falsos amigos)

17. Tarjetas de repaso ★

Tapa la columna de respuestas, contesta en voz alta y marca ☐ las que ya dominas. Repite las que no marcaste.

#	Pregunta	Respuesta	✔
1	¿Cuántas unidades y horas tiene el curso?	10 unidades de 3 horas: 30 horas en 5 días	☐
2	¿Cuál es el caso integrador?	Pedidos Legado (Delphi/DBISAM) → Pedidos Web (.NET y PostgreSQL)	☐
3	¿Las seis etapas del SDLC del programa?	Requisitos, Pruebas, Validación, Revisión, Evolución y Mantención	☐
4	¿Los tres estándares abiertos?	MCP, Agent Skills y AGENTS.md	☐
5	¿Las 4D?	Delegación, Descripción, Discernimiento y Diligencia	☐
6	¿Qué D es leer el diff y correr los tests?	Discernimiento	☐
7	¿Qué D es escribir una buena spec?	Descripción	☐
8	¿Qué D es decidir qué hace la IA y qué la persona?	Delegación	☐
9	¿Qué D es hacerse responsable y transparentar el uso de IA?	Diligencia	☐
10	¿Asistente o agente?	El asistente sugiere; el agente usa herramientas y ejecuta en varios pasos	☐
11	¿Haiku, Sonnet u Opus para diagnosticar un legado?	Opus	☐
12	¿Y para generar datos de prueba?	Haiku	☐
13	¿El ciclo del agente?	Reunir contexto → actuar → verificar	☐
14	¿Qué hace plan mode?	Lee y propone; no modifica nada	☐
15	¿`/context`, `/compact` y `/clear`?	Ver el uso del contexto, resumir, empezar de cero	☐
16	¿La cadena de archivos de SDD?	intent.md → spec.md → plan.md → tasks.md	☐
17	¿Qué va en spec.md?	Comportamiento, alcance y criterios de aceptación verificables	☐
18	¿Qué es «tú firmas»?	La persona responde por cada commit; revisa el diff con un checklist	☐
19	¿Pasos chicos?	Una tarea, un incremento, un commit	☐
20	¿Qué es rewind?	Volver la conversación y los archivos a un punto anterior	☐
21	¿Qué archivo define una Skill?	SKILL.md, con `name` y `description` en el frontmatter	☐
22	¿Skill o AGENTS.md para «nunca usar `SELECT *`»?	AGENTS.md (regla general, siempre vigente)	☐
23	¿Skill o AGENTS.md para «cómo documentar un módulo»?	Skill (procedimiento específico)	☐
24	¿Para qué sirven los subagentes de lectura?	Explorar con contexto propio y devolver solo la conclusión	☐
25	¿Cómo se valida una regla de negocio extraída?	Con su archivo y línea, verificada en el código	☐

#	Pregunta	Respuesta	✔
26	¿Las cuatro decisiones de la matriz?	Mantener, refactorizar, modernizar o reemplazar	☐
27	¿Qué es strangler fig?	Migrar por partes detrás de una fachada, hasta apagar el sistema viejo	☐
28	¿Qué es un golden master?	Salidas del sistema viejo guardadas; el nuevo debe reproducirlas	☐
29	¿`Currency` de Delphi en C#?	`decimal`	☐
30	¿Montos en PostgreSQL?	`numeric(p,s)`, nunca `float`	☐
31	¿Qué es un criterio de corte?	La condición medible para apagar el legado	☐
32	¿La pirámide de tests?	Muchos unitarios, menos de integración, pocos E2E	☐
33	¿Bug → ?	Test que falla → corrección	☐
34	¿Por qué cobertura ≠ calidad?	Ejecutar una línea no significa verificarla	☐
35	¿Las cuatro reglas del pipeline seguro?	Secretos, permisos mínimos, versiones fijas y timeouts	☐
36	¿Qué es claude-code-action?	La GitHub Action oficial que corre Claude Code en el pipeline	☐
37	¿Qué es el modo headless?	`claude -p`, sin conversación, desde un script o CI	☐
38	¿Regla en CI?	La IA propone, la persona decide	☐
39	¿AGENTS.md o REVIEW.md?	AGENTS.md para todo el trabajo; REVIEW.md para la revisión	☐
40	¿Qué se hace con cada hallazgo de la IA?	Se acepta, se discute o se descarta con argumento	☐
41	¿Por qué el revisor tiene contexto limpio?	Para revisar como un tercero que no vio cómo se escribió el código	☐
42	¿Qué es prompt injection?	Instrucciones maliciosas en contenido que lee el agente	☐
43	¿Las capas de defensa en profundidad?	allow/deny, sandbox, hooks y revisión humana	☐
44	¿Qué es la política ejecutable?	Reglas como configuración y hooks, probadas contra violaciones	☐
45	¿Qué settings no se pueden saltar?	La configuración administrada de la organización	☐
46	¿Qué empaqueta un plugin?	Skills, comandos, subagentes, hooks y servidores MCP	☐
47	¿Qué expone un servidor MCP?	Herramientas, recursos y prompts	☐
48	¿Las métricas DORA?	Frecuencia de despliegue, tiempo de entrega, tasa de fallas por cambio y tiempo de recuperación	☐
49	¿Fórmula del ROI?	(Beneficios − costos) / costos	☐
50	¿Qué es el análisis de sensibilidad?	Ver qué supuesto mueve más el resultado	☐

18. Llegar con ventaja: borradores de los primeros entregables ★

18.1 Borrador del mapa de uso de IA de nuestro equipo (U01)

Lectura propia, con las columnas que pide el taller para `docs/mapa-sdlc.md`. Ajústalo con lo que salga del paso 3 (`formatearFolio()`).

Etapa	Se delega a Claude	Artefacto versionado	Revisa una persona	Riesgo	Evento Scrum
Requisitos	Redactar el borrador de la spec y proponer criterios de aceptación	`spec.md`	Producto aprueba alcance y criterios	Reglas de negocio inventadas	Refinamiento
Diseño	Proponer un plan y dos alternativas en plan mode	`plan.md`	Arquitectura elige y registra la descartada	Diseño que no respeta las convenciones	Sprint Planning
Desarrollo	Implementar por tareas pequeñas, con Skills propias	Código y `tasks.md`	Quien desarrolla lee el diff completo	Cambios no pedidos, dependencias inventadas	Sprint (trabajo diario)
Pruebas	Generar tests (uno por criterio) y datos sintéticos	Tests	Quien desarrolla y QA verifican que prueben algo	Tests que siempre pasan; datos reales filtrados	Sprint
Revisión	Revisión previa con un subagente de solo lectura	Comentarios del PR	Líder decide sobre cada hallazgo y aprueba el PR	Aceptar hallazgos sin leerlos	Antes de la Sprint Review
Integración y despliegue	Diagnosticar fallas del pipeline y proponer arreglos	Pipeline (YAML)	Persona aprueba el paso a producción	Secretos o permisos excesivos en CI	Incremento terminado (DoD)
Evolución y mantención	Leer el legado, extraer reglas con archivo y línea, migrar por partes	Diagnóstico, golden master	Criterio de corte por componente	Cambiar comportamiento sin notarlo	Retrospectiva y backlog

18.2 Autodiagnóstico de madurez (U09)

Lectura propia, sobre el modelo de cinco niveles de §10.1.

Dimensión	Nivel estimado	Por qué
Gobernanza y seguridad	4	Controles implementados y probados haciéndolos fallar
Estandarización (instrucciones, Skills, settings)	3	Ficha, Skills y settings versionados en el repositorio
Integración con CI	2	Etapa diseñada pero sin ejecutar; sensor local en un solo componente
Medición (DORA, adopción, ROI)	1 a 2	Indicadores definidos en SGI-PR-040, sin medición real todavía

Dimensión	Nivel estimado	Por qué
Distribución (plugins, marketplace)	1	No existe; el rol de Platform Engineering no tiene persona asignada

18.3 Preguntas que nos pueden hacer

Pregunta	Respuesta sugerida
¿Cómo usan IA hoy?	Claude Code con cuenta empresarial, con desarrollo guiado por specs y controles técnicos (hooks, permisos, auditoría) que probamos haciéndolos fallar
¿Qué es lo más difícil?	Medir. Tenemos buenos controles, pero no tenemos métricas de adopción ni DORA
¿Tienen legado?	Sí: Rayen 2020, que migramos a .NET y PostgreSQL con strangler fig y criterio de corte por componente
¿Cómo protegen los datos clínicos?	Nunca entran: datos sintéticos, semáforo de información y hooks que bloquean RUN y credenciales

19. Preguntas para el instructor ★

Las marcadas con ★ no deberían quedar sin respuesta.

Herramienta y estándares

1. ★ Ya sabemos que `CLAUDE.md` apaga `AGENTS.md`. Para un equipo que ya tiene `CLAUDE.md` con estructura propia, ¿recomienda un `CLAUDE.md` que importe un `AGENTS.md` común, o migrar todo a `AGENTS.md`?
2. ¿Qué propone para `intent.md` que no cubra `spec.md`? ¿Cuándo se puede omitir?
3. ¿Cuál es el modelo de madurez de 5 niveles que usa y qué indicador define cada nivel?
4. ¿REVIEW.md es una convención del curso o la lee Claude Code de forma nativa?

Nuestro contexto

5. ★ Usamos **Azure DevOps y Azure Pipelines**, no GitHub. ¿Cómo se replica `claude-code-action` y la revisión automática? ¿Con `claude -p` en un paso del pipeline?
6. ¿Algún matiz para usar **TUnit** en vez de xUnit en los ejercicios?
7. ★ En salud (datos clínicos, Ley 20.584), ¿qué recomienda sobre el nivel de autonomía y el uso de MCP con bases de datos?
8. Para el strangler fig, ¿cómo aplicaría el golden master cuando el legado es un servidor con base de datos compartida y no un archivo local?

Gobernanza y medición

9. ★ ¿Qué dicen exactamente los términos del plan Team sobre retención y uso de datos? ¿Cómo se alinean con la política de Vela (registro de 5 años)?
10. ¿Qué mide el panel de Analytics de Claude Code y cómo se usa para el ROI?
11. ¿Qué otras empresas de Vela LATAM están en el grupo y qué tan avanzadas van? Podría haber oportunidades de compartir Skills y plugins.

20. Fuentes ○

Sitio del curso: ffelipecuevasc.github.io/curso-ia-desarrolladores (unidades, guías de taller, glosario y programa en PDF).

El programa (bibliografía oficial, consultada por el instructor el 01/11/2026)

- Anthropic Academy (academy.claude.com/courses): «AI Fluency: Framework & Foundations», «AI Fluency for builders», «Claude Code 101», «Claude Code in action», «Introduction to agent skills», «Introduction to subagents», «Introduction to MCP», «AI-native SDLC playbook».
- Documentación de Claude Code (code.claude.com/docs/en/): best-practices, common-workflows, memory, skills, sub-agents, hooks-guide, github-actions, code-review, security, data-usage, plugins/overview, mcp, analytics, costs, jetbrains.
- github.com/anthropics/claude-code-action · claude.com/solutions/code-modernization.
- Estándares abiertos: agents.md · agentskills.io · aaif.io.
- Libros: Osmani, *Beyond Vibe Coding* (O'Reilly, 2025); Martinelli, *Spec-Driven Development* (Apress, 2026).
- dora.dev · scrumguides.org · OWASP Top 10.

Si sobra tiempo: prioriza **best-practices**, **memory**, **skills** y **security**. Son las páginas que más alimentan los talleres.

Repositorio (contexto Rayen)

- `CLAUDE.md`: ficha y tabla de controles
- `.agents/harness/matriz-cumplimiento.md`
- `.agents/plans/plataforma-ia/sprint-10-plataforma-ia-interna.md` y specs de `.agents/specs/plataforma-ia/`
- `vela-ai-policy.md` · SGI-PR-040-01 Procedimiento Desarrollo Agéntico con Inteligencia Artificial · Glosario del marco (Base de conocimiento, 04 Glosario)

Notas libres